

Phase 4 Notes: Recursive Insertion and Recursive Deletion in a BST

1. Where You Are Now

Before Phase 4, you already learned the foundation of Binary Search Trees.

Phase 1: Binary Tree Basics

You learned that a binary tree is made of nodes. Each node has three parts:

```
struct Node {  
    struct Node *lchild;  
    int data;  
    struct Node *rchild;  
};
```

Meaning:

- `lchild` stores the address of the left child.
- `data` stores the value inside the node.
- `rchild` stores the address of the right child.
- `NULL` means there is no node in that direction.

You also learned:

- The **root** is the top node.
- A **leaf** is a node with no children.
- A **subtree** is a smaller tree inside a bigger tree.
- Recursion works well with trees because every child is also the root of a smaller tree.
- `height()` finds the longest downward path.

Phase 2: BST Searching

You learned the BST rule:

```
left subtree values < node value < right subtree values
```

When searching in a BST:

```
key == current node data  -> found
key < current node data   -> go left
key > current node data   -> go right
current pointer == NULL   -> not found
```

The most important idea was:

BST search follows only one path, not the whole tree.

Phase 3: Iterative Insertion

You learned that insertion is like searching first.

To insert a value:

1. Start from the root.
2. Compare the new key with the current node.
3. Move left or right using the BST rule.
4. If the key already exists, stop.
5. If you reach `NULL`, insert the new node there.

In iterative insertion, you used two pointers:

```
t = current pointer;
r = parent pointer;
```

The pointer `t` moves down the tree. The pointer `r` remembers the parent of `t`.

This is needed because when `t` becomes `NULL`, you cannot attach a new node to `t`. You need the parent pointer `r`.

2. Main Goal of Phase 4

Phase 4 teaches two important BST operations:

1. **Recursive insertion**
2. **Recursive deletion**

Recursive insertion is another way to insert a new value into a BST.

Recursive deletion is more difficult because deleting a node can break the tree if we are not careful.

After deleting a node, the tree must still follow the BST rule:

```
left subtree values < node value < right subtree values
```

3. Big Idea of Phase 4

The biggest idea in Phase 4 is this:

```
Recursive insertion and recursive deletion both return the updated subtree root.
```

That means every recursive call gives back the correct root of the subtree after the operation is done.

So the parent must save that returned pointer.

Example:

```
p->lchild = RInsert(p->lchild, key);
```

This means:

```
Insert the key into my left subtree.  
Then reconnect my left child to the updated left subtree.
```

For deletion:

```
p->lchild = Delete(p->lchild, key);
```

This means:

```
Delete the key from my left subtree.  
Then reconnect my left child to the updated left subtree.
```

This is the heart of Phase 4.

Part A: Recursive Insertion

4. What Is Recursive Insertion?

Recursive insertion inserts a new key into a BST using recursion instead of a loop.

It still follows the same BST rule:

```
smaller key -> go left  
larger key  -> go right  
equal key   -> do not insert duplicate
```

The main difference is how the parent link is handled.

In iterative insertion, we used a parent pointer `r`.

In recursive insertion, we do not need `r` because recursion automatically returns to the parent call.

5. Iterative Insertion vs Recursive Insertion

Iterative insertion

Iterative insertion uses:

```
t = current pointer;  
r = parent pointer;
```

When `t` becomes `NULL`, `r` tells us where to attach the new node.

Recursive insertion

Recursive insertion uses return values.

Instead of remembering the parent manually, the recursive call returns to its caller. That caller is the parent.

So recursive insertion uses lines like:

```
p->lchild = RInsert(p->lchild, key);
```

and:

```
p->rchild = RInsert(p->rchild, key);
```

These lines reconnect the tree after insertion.

6. Recursive Insertion Mental Model

Think of recursive insertion like this:

```
If the current pointer is NULL:
    create a new node and return it

Otherwise:
    if the key is smaller:
        insert into the left subtree
    if the key is larger:
        insert into the right subtree
    return the current node
```

The new node is created only when the function reaches `NULL`.

That `NULL` is the correct empty place where the new node belongs.

7. Recursive Insertion Code

```
struct Node* RInsert(struct Node *p, int key) {
    struct Node *t = NULL;

    if (p == NULL) {
        t = (struct Node*)malloc(sizeof(struct Node));
        t->data = key;
        t->lchild = NULL;
        t->rchild = NULL;
        return t;
    }

    if (key < p->data) {
        p->lchild = RInsert(p->lchild, key);
    }
    else if (key > p->data) {
        p->rchild = RInsert(p->rchild, key);
    }
}
```

```
    return p;  
}
```

8. Recursive Insertion Code Explanation

Function header

```
struct Node* RInsert(struct Node *p, int key)
```

This function returns a pointer to a `Node`.

It returns the root of the subtree after insertion.

Parameters:

- `p` is the current subtree root.
- `key` is the value we want to insert.

Base case

```
if (p == NULL)
```

This means:

We found the empty place where the new key belongs.

So we create a new node.

Creating the new node

```
t = (struct Node*)malloc(sizeof(struct Node));  
t->data = key;  
t->lchild = NULL;  
t->rchild = NULL;  
return t;
```

This creates memory for the new node.

Then:

- `t->data = key` stores the value.
 - `t->lchild = NULL` means the new node has no left child yet.
 - `t->rchild = NULL` means the new node has no right child yet.
 - `return t` sends the new node address back to the parent call.
-

Going left

```
if (key < p->data) {  
    p->lchild = RInsert(p->lchild, key);  
}
```

This means:

The key is smaller than the current node.
So it belongs somewhere in the left subtree.

The important part is:

```
p->lchild =
```

This reconnects the left child after insertion.

Going right

```
else if (key > p->data) {  
    p->rchild = RInsert(p->rchild, key);  
}
```

This means:

The key is larger than the current node.
So it belongs somewhere in the right subtree.

The important part is:

```
p->rchild =
```

This reconnects the right child after insertion.

Returning current node

```
return p;
```

This means:

```
After insertion is done, return the current subtree root.
```

This helps the caller reconnect its child pointer correctly.

9. Why `root = RInsert(root, key)` Is Important

When calling recursive insertion, always save the return value:

```
root = RInsert(root, key);
```

Do not just write:

```
RInsert(root, key);
```

Why?

Because if the tree is empty, `root` is `NULL`.

The function creates a new node and returns its address.

If you do not save that address in `root`, the tree will still be empty.

Correct:

```
root = RInsert(root, 30);
```

This makes `30` the root if the tree was empty.

10. Duplicate Handling in Recursive Insertion

The recursive insertion code has this condition:

```
if (key < p->data)
```

and:

```
else if (key > p->data)
```

Notice that there is no case for:

```
key == p->data
```

That means duplicate values are ignored.

If the key is already in the tree, the function simply returns `p` without creating a new node.

So this BST does not insert duplicates.

11. Dry Run 1: Insert 30, 20, 25

Start with an empty tree.

```
root = NULL
```

Insert 30

Call:

```
root = RInsert(root, 30);
```

Since `root` is `NULL`, create node `30`.

Tree becomes:

30

Insert 20

Call:

```
root = RInsert(root, 20);
```

Current node is 30.

Compare:

```
20 < 30
```

So go left.

Left child of 30 is NULL, so create node 20.

Tree becomes:

```
  30
 /
20
```

The important link is:

```
30->lchild = 20;
```

Insert 25

Call:

```
root = RInsert(root, 25);
```

Current node is 30.

Compare:

25 < 30

Go left to 20 .

Compare:

25 > 20

Go right.

Right child of 20 is NULL , so create node 25 .

Tree becomes:

```
  30
 /
20
  25
```

The important links are:

```
20->rchild = 25;
30->lchild = 20;
```

12. Dry Run 2: Insert 10, 5, 20, 8, 30

Start with an empty tree.

Keys:

10, 5, 20, 8, 30

Insert 10

Tree is empty, so 10 becomes root.

10

Insert 5

Compare:

$5 < 10$

So 5 becomes left child of 10.

```
  10
 /
5
```

Insert 20

Compare:

$20 > 10$

So 20 becomes right child of 10.

```
  10
 /  \
5    20
```

Insert 8

Compare with 10:

$8 < 10$

Go left to 5.

Compare with 5:

$8 > 5$

So 8 becomes right child of 5.

```
    10
   /
  5   20
   \
    8
```

Insert 30

Compare with 10:

30 > 10

Go right to 20.

Compare with 20:

30 > 20

So 30 becomes right child of 20.

Final tree:

```
    10
   /
  5   20
   \   \
    8   30
```

Part B: Recursive Deletion

13. What Is Recursive Deletion?

Recursive deletion removes a key from a BST using recursion.

Deletion starts like searching.

First, find the node.

```
if key < current node data:
    go left

if key > current node data:
    go right

if key == current node data:
    this is the node to delete
```

After the node is found, we must delete it carefully.

The tree must still be a valid BST after deletion.

14. Why Deletion Is Harder Than Insertion

Insertion only adds a new node at a `NULL` position.

Deletion removes an existing node.

Removing a node can create problems:

- The parent may lose a child pointer.
- A subtree may become disconnected.
- The BST order may break.
- A node with two children cannot simply be freed.

That is why deletion has different cases.

15. The Three Deletion Cases

There are three main deletion cases.

Case	Node has	What we do
Case 1	No children	Delete the node and return <code>NULL</code>
Case 2	One child	Delete the node and return its child
Case 3	Two children	Replace the node using predecessor or successor, then delete the duplicate

Case 1: Deleting a Leaf Node

16. What Is a Leaf Node?

A leaf node is a node with no children.

That means:

```
p->lchild == NULL && p->rchild == NULL
```

Example:

```
  10
 /
5   20
```

Here, `5` and `20` are leaf nodes.

17. How to Delete a Leaf Node

If the node has no children:

1. Free the node.
2. Return `NULL`.

Code:

```
if (p->lchild == NULL && p->rchild == NULL) {
    free(p);
    return NULL;
}
```

Why return `NULL`?

Because the parent should no longer point to this node.

Example:

```
p->lchild = Delete(p->lchild, key);
```

If the deleted node was the left child, the returned `NULL` makes the parent's left child become `NULL`.

18. Dry Run: Delete Leaf Node 5

Tree:

```
  10
 /
5   20
```

Delete `5`.

Steps:

1. Start at `10`.
2. `5 < 10`, so go left.
3. Find node `5`.
4. Node `5` has no children.
5. Free node `5`.
6. Return `NULL`.
7. `10->lchild` becomes `NULL`.

Result:

```
  10

  20
```

Case 2: Deleting a Node With One Child

19. What Is the One-Child Case?

A node has one child if exactly one of its child pointers is not `NULL`.

Example:

```
  10

  20
```


30

Node 20 has one child: 30.

20. How to Delete a Node With One Child

If the node has only one child, we do not want to lose that child.

Instead:

1. Save the child pointer.
2. Free the node.
3. Return the child pointer.

That child will connect directly to the deleted node's parent.

21. One Child on the Right

If the node has no left child but has a right child:

```
else if (p->lchild == NULL) {  
    q = p->rchild;  
    free(p);  
    return q;  
}
```

Meaning:

The right child replaces the deleted node.

22. One Child on the Left

If the node has no right child but has a left child:

```
else if (p->rchild == NULL) {  
    q = p->lchild;  
    free(p);
```

```
    return q;  
}
```

Meaning:

The left child replaces the deleted node.

23. Dry Run: Delete Node 20 With One Child

Tree:

```
  10  
     
  20  
     
  30
```

Delete 20.

Steps:

1. Start at 10.
2. $20 > 10$, so go right.
3. Find node 20.
4. Node 20 has one child: 30.
5. Save pointer to 30.
6. Free node 20.
7. Return 30.
8. `10->rchild` becomes 30.

Result:

```
  10  
     
  30
```

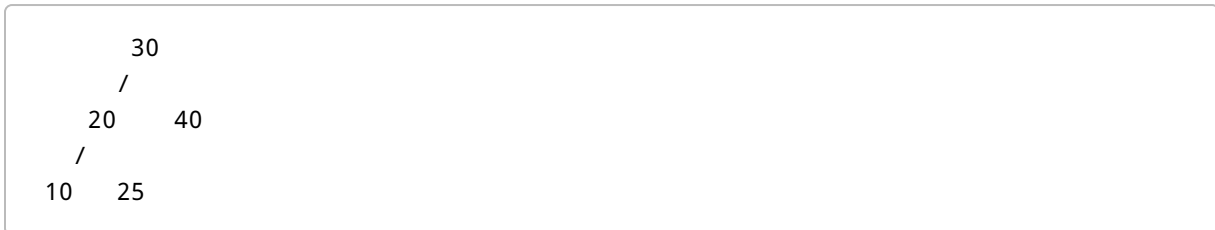
This keeps the BST correct.

Case 3: Deleting a Node With Two Children

24. Why the Two-Child Case Is Hard

A node with two children has both a left subtree and a right subtree.

Example:



Delete 30.

We cannot simply free 30 because then the left subtree and right subtree would lose their connection.

So we replace 30 with a value that keeps the BST rule correct.

The replacement can be either:

1. Inorder predecessor
2. Inorder successor

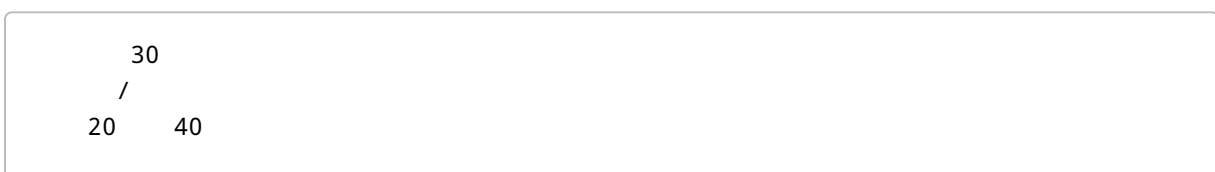
25. Inorder Predecessor

The inorder predecessor is the largest value in the left subtree.

To find it:

Go to the left subtree.
Then keep going right until you cannot go right anymore.

Example:



```
  /
10 25
```

The left subtree of 30 is:

```
  20
 /
10 25
```

The largest value in this subtree is 25.

So the inorder predecessor of 30 is 25.

26. Inorder Successor

The inorder successor is the smallest value in the right subtree.

To find it:

Go to the right subtree.
Then keep going left until you cannot go left anymore.

Example:

```
  30
 /
20 40
   /
  35
```

The right subtree of 30 is:

```
  40
 /
35
```

The smallest value in this subtree is 35.

So the inorder successor of 30 is 35.

27. Why Predecessor or Successor Works

When deleting a node with two children, we need a replacement that keeps the sorted order.

The predecessor works because it is:

smaller than the deleted value
but the largest available smaller value

The successor works because it is:

larger than the deleted value
but the smallest available larger value

So both can preserve the BST rule.

28. The Taller-Side Replacement Rule

One version of BST deletion chooses the replacement based on height.

If the left subtree is taller:

Use inorder predecessor.

If the right subtree is taller or equal:

Use inorder successor.

Why?

Because choosing from the taller side may help avoid making the tree shape worse.

This does not fully balance the tree like an AVL tree, but it is a useful classroom rule.

29. Two-Child Deletion Process

If the node has two children:

1. Choose predecessor or successor.

2. Copy the replacement value into the node being deleted.
3. Now the replacement value appears twice.
4. Recursively delete the original replacement node from its old place.

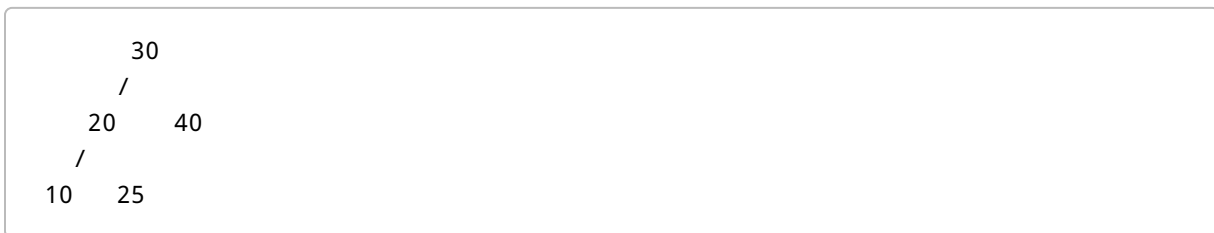
This is very important.

We do not usually move the whole replacement node physically.

Instead, we copy its data.

30. Dry Run: Delete 30 Using Predecessor

Tree:



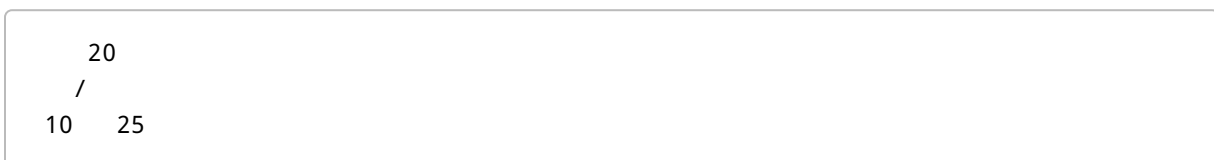
Delete 30.

Step 1: Find node 30.

Step 2: 30 has two children.

Step 3: Choose predecessor from the left subtree.

Left subtree:

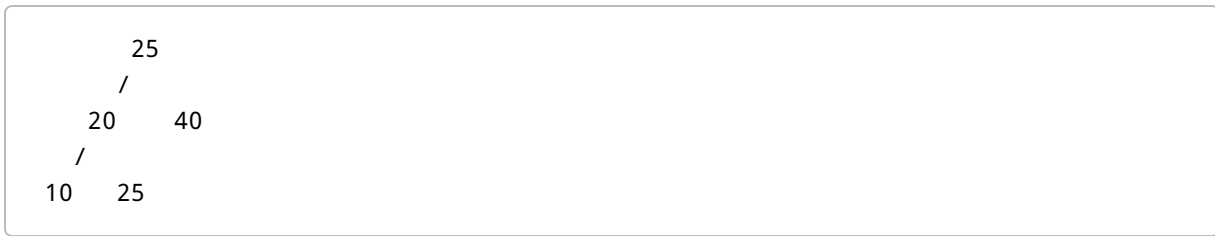


Largest value is 25.

So predecessor is 25.

Step 4: Copy 25 into the node that had 30.

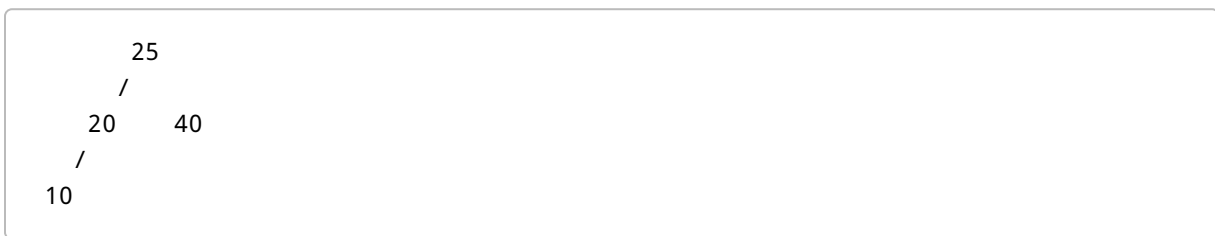
Temporary tree:



Now there are two 25 s.

Step 5: Delete the original 25 from the left subtree.

Final tree:



The BST is still valid.

31. Cascading Deletion

Cascading deletion means deletion may continue downward after a replacement.

This happens in the two-child case.

When we copy predecessor or successor data, the replacement value still exists in its old position.

So we must delete the original replacement node.

Sometimes that original replacement node may also have a child.

So the deletion continues until it reaches a simpler case:

- leaf case, or
- one-child case

This chain reaction is called cascading deletion.

32. Cascading Deletion Example

Imagine a left-skewed tree:

```
50
 /
40
 /
30
```

Delete 50 .

Step 1: 50 has a left child.

Step 2: Use predecessor from the left side.

The predecessor is 40 .

Step 3: Copy 40 into the root.

Temporary idea:

```
40
 /
40
 /
30
```

Now there are two 40 s.

Step 4: Delete the old 40 from the left subtree.

Old 40 has one child: 30 .

So 30 replaces old 40 .

Final tree:

```
40
 /
30
```


The deletion cascaded downward.

Part C: Helper Functions for Deletion

33. Why Helper Functions Are Needed

Recursive deletion uses helper functions to make the code cleaner.

The main helper functions are:

1. Height()
2. InPre()
3. InSucc()

They help with the two-child deletion case.

34. Height Function

The Height() function finds the height of a subtree.

```
int Height(struct Node *p) {  
    int x, y;  
  
    if (p == NULL) {  
        return 0;  
    }  
  
    x = Height(p->lchild);  
    y = Height(p->rchild);  
  
    return x > y ? x + 1 : y + 1;  
}
```

Meaning

If the subtree is empty:

```
height = 0
```

Otherwise:

```
height = taller side + 1
```

This function is used to decide whether the left subtree or right subtree is taller.

35. InPre Function

`InPre()` finds the inorder predecessor.

The inorder predecessor is the rightmost node in the left subtree.

```
struct Node* InPre(struct Node *p) {  
    while (p != NULL && p->rchild != NULL) {  
        p = p->rchild;  
    }  
  
    return p;  
}
```

Meaning

This function keeps moving right.

Why right?

Because larger values are on the right in a BST.

So the rightmost node is the largest value in that subtree.

36. InSucc Function

`InSucc()` finds the inorder successor.

The inorder successor is the leftmost node in the right subtree.

```
struct Node* InSucc(struct Node *p) {  
    while (p != NULL && p->lchild != NULL) {  
        p = p->lchild;  
    }  
}
```

```
    return p;
}
```

Meaning

This function keeps moving left.

Why left?

Because smaller values are on the left in a BST.

So the leftmost node is the smallest value in that subtree.

Part D: Recursive Delete Function

37. Recursive Delete Code

```
struct Node* Delete(struct Node *p, int key) {
    struct Node *q;

    if (p == NULL) {
        return NULL;
    }

    if (key < p->data) {
        p->lchild = Delete(p->lchild, key);
    }
    else if (key > p->data) {
        p->rchild = Delete(p->rchild, key);
    }
    else {
        if (p->lchild == NULL && p->rchild == NULL) {
            free(p);
            return NULL;
        }
        else if (p->lchild == NULL) {
            q = p->rchild;
            free(p);
            return q;
        }
        else if (p->rchild == NULL) {
            q = p->lchild;
            free(p);
        }
    }
}
```

```

        return q;
    }
    else if (Height(p->lchild) > Height(p->rchild)) {
        q = InPre(p->lchild);
        p->data = q->data;
        p->lchild = Delete(p->lchild, q->data);
    }
    else {
        q = InSucc(p->rchild);
        p->data = q->data;
        p->rchild = Delete(p->rchild, q->data);
    }
}

return p;
}

```

38. Delete Function Explanation

Function header

```
struct Node* Delete(struct Node *p, int key)
```

This function returns the updated root of the current subtree.

Parameters:

- `p` is the current subtree root.
- `key` is the value to delete.

If the tree is empty

```

if (p == NULL) {
    return NULL;
}

```

This means:

The key was not found.
There is nothing to delete.

So return `NULL`.

Search left

```
if (key < p->data) {  
    p->lchild = Delete(p->lchild, key);  
}
```

If the key is smaller than the current node, it must be in the left subtree.

After deletion, reconnect the left child.

Search right

```
else if (key > p->data) {  
    p->rchild = Delete(p->rchild, key);  
}
```

If the key is larger than the current node, it must be in the right subtree.

After deletion, reconnect the right child.

Node found

```
else {
```

This means:

```
key == p->data
```

So this is the node we need to delete.

Now we handle the three deletion cases.

Leaf case

```
if (p->lchild == NULL && p->rchild == NULL) {  
    free(p);  
    return NULL;  
}
```

The node has no children.

Delete it and return `NULL`.

One right child

```
else if (p->lchild == NULL) {  
    q = p->rchild;  
    free(p);  
    return q;  
}
```

The node has no left child.

So the right child replaces it.

One left child

```
else if (p->rchild == NULL) {  
    q = p->lchild;  
    free(p);  
    return q;  
}
```

The node has no right child.

So the left child replaces it.

Two children: use predecessor

```
else if (Height(p->lchild) > Height(p->rchild)) {  
    q = InPre(p->lchild);
```

```
p->data = q->data;  
p->lchild = Delete(p->lchild, q->data);  
}
```

If the left subtree is taller, use the inorder predecessor.

Steps:

1. Find predecessor `q`.
2. Copy `q->data` into `p->data`.
3. Delete the original `q->data` from the left subtree.

Two children: use successor

```
else {  
    q = InSucc(p->rchild);  
    p->data = q->data;  
    p->rchild = Delete(p->rchild, q->data);  
}
```

If the right subtree is taller or equal, use the inorder successor.

Steps:

1. Find successor `q`.
2. Copy `q->data` into `p->data`.
3. Delete the original `q->data` from the right subtree.

Return current node

```
return p;
```

This returns the updated subtree root to the caller.

Part E: Full Phase 4 Program

39. Complete C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    struct Node *lchild;
    int data;
    struct Node *rchild;
};

struct Node *root = NULL;

struct Node* createNode(int key) {
    struct Node *p = (struct Node*)malloc(sizeof(struct Node));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }

    p->data = key;
    p->lchild = NULL;
    p->rchild = NULL;

    return p;
}

struct Node* RInsert(struct Node *p, int key) {
    if (p == NULL) {
        return createNode(key);
    }

    if (key < p->data) {
        p->lchild = RInsert(p->lchild, key);
    }
    else if (key > p->data) {
        p->rchild = RInsert(p->rchild, key);
    }

    return p;
}

int Height(struct Node *p) {
```



```

    int x, y;

    if (p == NULL) {
        return 0;
    }

    x = Height(p->lchild);
    y = Height(p->rchild);

    return x > y ? x + 1 : y + 1;
}

struct Node* InPre(struct Node *p) {
    while (p != NULL && p->rchild != NULL) {
        p = p->rchild;
    }

    return p;
}

struct Node* InSucc(struct Node *p) {
    while (p != NULL && p->lchild != NULL) {
        p = p->lchild;
    }

    return p;
}

struct Node* Delete(struct Node *p, int key) {
    struct Node *q;

    if (p == NULL) {
        return NULL;
    }

    if (key < p->data) {
        p->lchild = Delete(p->lchild, key);
    }
    else if (key > p->data) {
        p->rchild = Delete(p->rchild, key);
    }
    else {
        if (p->lchild == NULL && p->rchild == NULL) {
            free(p);
            return NULL;
        }
        else if (p->lchild == NULL) {
            q = p->rchild;

```

```

        free(p);
        return q;
    }
    else if (p->rchild == NULL) {
        q = p->lchild;
        free(p);
        return q;
    }
    else if (Height(p->lchild) > Height(p->rchild)) {
        q = InPre(p->lchild);
        p->data = q->data;
        p->lchild = Delete(p->lchild, q->data);
    }
    else {
        q = InSucc(p->rchild);
        p->data = q->data;
        p->rchild = Delete(p->rchild, q->data);
    }
}

return p;
}

void inorder(struct Node *p) {
    if (p != NULL) {
        inorder(p->lchild);
        printf("%d ", p->data);
        inorder(p->rchild);
    }
}

int main() {
    int keys[] = {10, 5, 20, 8, 30};
    int n = sizeof(keys) / sizeof(keys[0]);

    for (int i = 0; i < n; i++) {
        root = RInsert(root, keys[i]);
    }

    printf("Before deletion: ");
    inorder(root);
    printf("\n");

    root = Delete(root, 20);

    printf("After deleting 20: ");
    inorder(root);
    printf("\n");
}

```

```
    return 0;  
}
```

40. Program Output

Before deletion: 5 8 10 20 30
After deleting 20: 5 8 10 30

41. Program Tree Before Deletion

The inserted keys are:

10, 5, 20, 8, 30

The tree becomes:

```
    10  
   /  
  5   20  
   \  
  8   30
```

In-order traversal prints:

5 8 10 20 30

This is sorted, so the BST is valid.

42. What Happens When Deleting 20?

Node 20 has one child: 30.

So this is the one-child deletion case.

The code does this:

```
q = p->rchild;  
free(p);  
return q;
```

Meaning:

```
Save 30.  
Delete 20.  
Return 30 so 10 can point to 30.
```

Final tree:

```
    10  
   /\n  5  30  
 /\n8
```

In-order traversal after deletion:

```
5 8 10 30
```

Still sorted, so the BST is still valid.

Part F: Complexity

43. Height Controls BST Operation Time

Most BST operations depend on height.

Let:

```
h = height of the tree
```

Then searching, inserting, and deleting usually follow one path from the root downward.

So their time is based on `h`.

44. Recursive Insertion Complexity

Recursive insertion follows one path from root to a `NULL` position.

General time:

$O(h)$

If the tree is balanced:

$O(\log n)$

If the tree is skewed:

$O(n)$

Extra space:

$O(h)$

Why?

Because recursion uses the call stack.

45. Recursive Deletion Complexity

Deletion first searches for the node.

That search takes:

$O(h)$

If the node has two children, deletion also finds predecessor or successor.

That also depends on height.

So standard deletion is usually explained as:

$O(h)$

Balanced tree:

$O(\log n)$

Skewed tree:

$O(n)$

Extra space for recursive deletion:

$O(h)$

46. Height Helper Cost

The version of deletion shown here calls `Height()` to decide whether to use predecessor or successor.

`Height()` may visit many nodes in a subtree.

So this classroom version can do extra work because of repeated height calculations.

The simple way to remember:

Basic BST operation cost depends on height.
But calling `Height()` inside deletion can add extra scanning work.

47. Complexity Table

Operation	General Time	Balanced Tree	Skewed Tree	Extra Space
Recursive insert	$O(h)$	$O(\log n)$	$O(n)$	$O(h)$
Search part of delete	$O(h)$	$O(\log n)$	$O(n)$	$O(h)$
InPre / InSucc	$O(h)$	$O(\log n)$	$O(n)$	$O(1)$ for the loop itself

Operation	General Time	Balanced Tree	Skewed Tree	Extra Space
Recursive delete	Usually taught as $O(h)$	$O(\log n)$	$O(n)$	$O(h)$
Height helper	$O(n)$ for checked subtree	Extra work possible	Extra work possible	$O(h)$

Part G: Common Beginner Mistakes

48. Mistake 1: Forgetting to Save the Return Value

Wrong:

```
RInsert(root, 10);
```

Correct:

```
root = RInsert(root, 10);
```

Also for deletion:

```
root = Delete(root, 20);
```

Why?

Because recursive functions return the updated subtree root.

49. Mistake 2: Thinking Recursion Automatically Updates Links

Recursion does not automatically reconnect child pointers.

Wrong idea:

```
I called the function, so the tree is automatically fixed.
```

Correct idea:

```
p->lchild = RInsert(p->lchild, key);
```

and:

```
p->lchild = Delete(p->lchild, key);
```

The assignment is what reconnects the tree.

50. Mistake 3: Forgetting the `NULL` Base Case

Every recursive tree function needs a base case.

For insertion:

```
if (p == NULL) {  
    create and return new node;  
}
```

For deletion:

```
if (p == NULL) {  
    return NULL;  
}
```

Without a `NULL` base case, the code may crash.

51. Mistake 4: Inserting Duplicates Accidentally

The Phase 4 insertion code ignores duplicates.

If:

```
key == p->data
```

then the function does not insert a new node.

This keeps the BST simple.

52. Mistake 5: Deleting a Two-Child Node Directly

Wrong idea:

```
If I find the node, just free it.
```

This is only safe for a leaf node.

If the node has two children, freeing it directly can disconnect the tree.

Correct idea:

```
Replace it with predecessor or successor.  
Then delete the original replacement node.
```

53. Mistake 6: Forgetting to Delete the Duplicate Replacement

When deleting a two-child node, we copy predecessor or successor data.

Example:

```
30 becomes 25
```

But the original `25` still exists in the tree.

So we must delete the original `25` recursively.

54. Mistake 7: Confusing Predecessor and Successor

Remember:

```
Predecessor = largest value in the left subtree  
Successor   = smallest value in the right subtree
```

Another way:

Predecessor: go left, then far right
Successor: go right, then far left

55. Mistake 8: Saying BST Operations Are Always $O(\log n)$

Wrong:

BST insertion and deletion are always $O(\log n)$.

Correct:

BST operations are $O(h)$.

If the tree is balanced:

$O(h) = O(\log n)$

If the tree is skewed:

$O(h) = O(n)$

Part H: Key Rules to Remember

56. Recursive Insertion Rules

```
If p is NULL:
    create a new node and return it

If key is smaller:
    p->lchild = RInsert(p->lchild, key)

If key is larger:
    p->rchild = RInsert(p->rchild, key)

Return p
```

57. Recursive Deletion Rules

```
If p is NULL:
    return NULL

If key is smaller:
    p->lchild = Delete(p->lchild, key)

If key is larger:
    p->rchild = Delete(p->rchild, key)

If key is found:
    handle leaf, one-child, or two-child case

Return p
```

58. Deletion Case Rules

```
No children:
    free node
    return NULL

Only right child:
    save right child
    free node
    return right child

Only left child:
    save left child
    free node
    return left child

Two children:
    choose predecessor or successor
    copy replacement data
    delete original replacement node
```

59. Predecessor and Successor Rules

Inorder predecessor:
largest value in left subtree
go left, then far right

Inorder successor:
smallest value in right subtree
go right, then far left

Part I: Mini Practice

60. Practice 1: Recursive Insert

Insert these keys:

50, 30, 70, 20, 40

Resulting BST:

```
      50
     /
    30   70
   /
  20   40
```

In-order traversal:

20 30 40 50 70

61. Practice 2: Delete a Leaf

Tree:

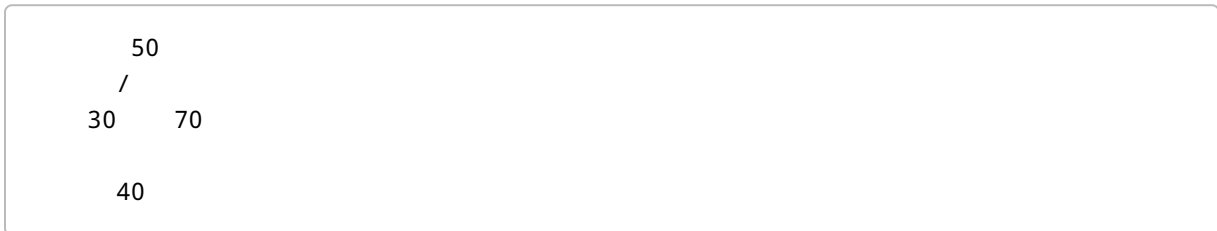
```
      50
     /
    30   70
```



Delete 20.

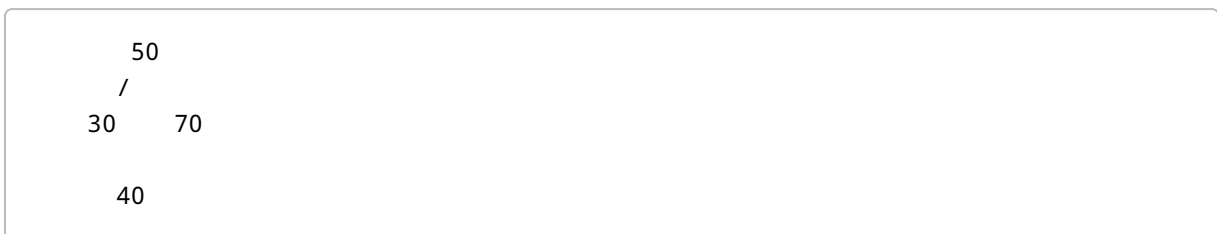
20 is a leaf.

Result:



62. Practice 3: Delete a Node With One Child

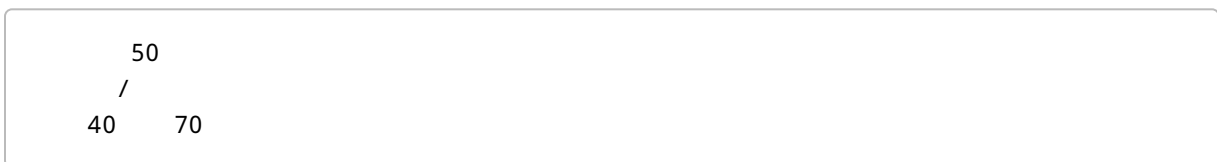
Tree:



Delete 30.

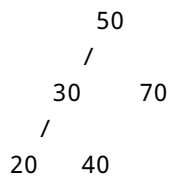
30 has one child: 40.

Result:



63. Practice 4: Delete a Node With Two Children

Tree:



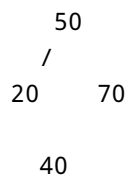
Delete 30.

Node 30 has two children: 20 and 40.

Using predecessor:

- Predecessor is 20.
- Copy 20 into 30.
- Delete original 20.

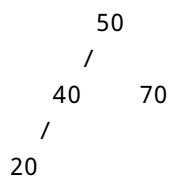
Result:



Using successor:

- Successor is 40.
- Copy 40 into 30.
- Delete original 40.

Result:



Both results are valid BSTs.

Part J: Final Summary

64. What You Learned in Phase 4

You learned recursive insertion and recursive deletion in a BST.

Recursive insertion:

- Uses the BST rule.
- Creates a node when it reaches `NULL`.
- Returns the new or unchanged subtree root.
- Reconnects links using assignment.

Recursive deletion:

- Searches for the key first.
- Deletes using one of three cases.
- Returns the updated subtree root.
- Reconnects links after deletion.

Deletion cases:

1. Leaf node: delete and return `NULL`.
2. One child: delete and return the child.
3. Two children: replace using predecessor or successor, then delete the original replacement.

Helper functions:

- `Height()` finds subtree height.
- `InPre()` finds the largest value in the left subtree.
- `InSucc()` finds the smallest value in the right subtree.

Complexity:

- General BST operation time is $O(h)$.
- Balanced tree time is $O(\log n)$.
- Skewed tree time is $O(n)$.
- Recursive operations use $O(h)$ stack space.

65. Final Mental Model

For recursive insertion, think:

Go down until NULL.
Create the node.
Return it upward.
Parents reconnect their child pointers.

For recursive deletion, think:

Search for the node.
When found, check how many children it has.

No children:
 delete and return NULL

One child:
 delete and return the child

Two children:
 copy predecessor or successor
 delete the original replacement node

The most important sentence in Phase 4 is:

Every recursive insert or delete call returns the updated root of that subtree.

If you understand that, Phase 4 becomes much easier.